Language Modeling

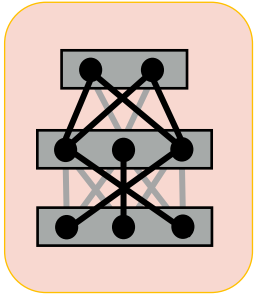
CSCI 601-771 (NLP: Self-Supervised Models)

https://self-supervised.cs.jhu.edu/fa2026/

Recap: Self-Supervised Models

- Earlier we define **Self-Supervised** models as as predictive models of the world!

“Wings over Kansas is [MASK]”



“Wings over Kansas is an aviation website founded in 1998 by Carl Chance owned by Chance Communications, Inc.”

Language Modeling: Motivation

- Earlier we define **Self-Supervised** models as as predictive models of the world!

- **Language models** are self-supervised, or predictive models of language.

- How do you formulate? How do you build them?

Language Modeling: Definitions and History

The

The cat

The cat sat

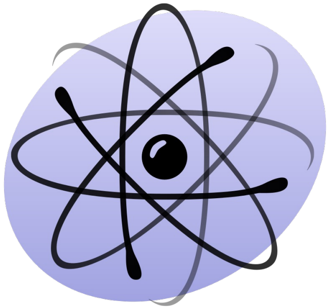
The cat sat on

The cat sat on __?__

The cat sat on __?__

The cat sat on __?__



The cat sat on __?__

$$\mathbf{P}(\underbrace{\text{mat}}_{\text{next word}} \mid \underbrace{\text{The cat sat on the}}_{\text{context or prefix}})$$

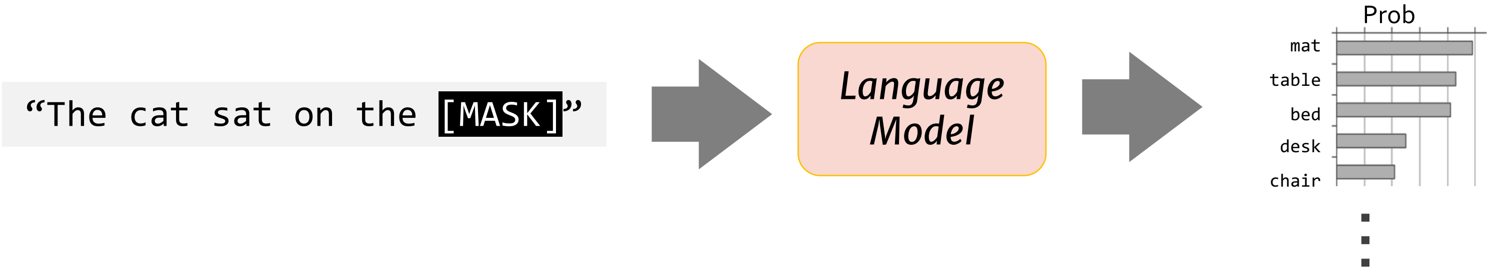
Probability of Upcoming Word

$$\mathbf{P}(\underbrace{X_t}_{\text{next word}} \mid \underbrace{X_1, \ldots, X_{t-1}}_{\text{context or prefix}})$$

LMs as a Marginal Distribution

- Directly we train models on "marginals":

$$\mathbf{P}(\overbrace{X_t}^{\text{next word}} | \overbrace{X_1, \dots, X_{t-1}}^{\text{context}})$$



LMs as Implicit Joint Distribution over Language

- While language modeling involves learning the marginals, we are implicitly learning the full/joint distribution of language.

    - Remember the chain rule:

$$\mathbf{P}(X_1, \ldots, X_t) = \mathrm{P}(X_1) \prod_{i=1}^{t} \mathrm{P}(X_i \mid X_1, X_2 \ldots, X_{i-1})$$

- Language Modeling ≜ learning prob distribution over language sequence.

The Language Modeling Problem, Formally

- Learning to assign a probability to every sequence:
  - **Finite** vocabulary $\Sigma = \{x_1, x_2, \ x_3, \dots, x_n\}$
  - **Infinite** set of sequences in this language $\Sigma^* = \{x_1, x_1x_2, x_2x_3, x_2x_1x_3, x_4x_3x_1x_2, \dots\}$

$$\sum_{e \in \Sigma^*} p_{\mathrm{LM}}(e) = 1,$$
$$p_{\mathrm{LM}}(e) \geq 0, \forall e \in \Sigma^*$$

Note that this is a (joint) distribution over sequences; different from (but related to) the conditional.

- Note, this statement does not specify the vocabulary (the atomic unit).
  - Example units: words, characters, bytes, etc.

Suppose we have a language model now.
What can we do with it?

Doing Things with Language Model

- What is the probability of ….

“I like Johns Hopkins University”

“like Hopkins I University Johns”

Doing Things with Language Model

- What is the probability of ….

"I like Johns Hopkins University"

"like Hopkins I University Johns"

- LMs assign a probability to every sentence (or any string of words).

P("I like Johns Hopkins University") $=10^{-5}$

P("like Hopkins I University Johns") $=10^{-15}$

Doing Things with Language Model (2)

- We can rank sentences.

$$\mathbf{P}(\underbrace{X_t}_{\text{next word}} \mid \underbrace{X_1, \ldots, X_{t-1}}_{\text{context}})$$

- The probability is an indicator of how *familiar* the sequence is to the LM.
  - Grammaticality, fluency, factuality, etc.

P(*"I like Johns Hopkins University."*) > **P**(*"I like John Hopkins University"*)

P(*"I like Johns Hopkins University."*) > **P**(*"University. I Johns EOS Hopkins like"*)

P(*"JHU is located in Baltimore."*) > **P**(*"JHU is located in Virginia."*)

Doing Things with Language Model (3)

- Can also generate strings!

$$\mathbf{P}(\underbrace{X_t}_{\text{next word}} | \underbrace{X_1, \ldots, X_{t-1}}_{\text{context}})$$

- Let's say we start *"Johns Hopkins is "*
- Using this prompt as an initial condition, recursively sample from an LM:

1. Sample from **P**(X | *"Johns Hopkins is "*) → "located"
2. Sample from **P**(X | *"Johns Hopkins is located"*) → "at"
3. Sample from **P**(X | *"Johns Hopkins is located at"*) → "the"
4. Sample from **P**(X | *"Johns Hopkins is located at the"*) → "state"
5. Sample from **P**(X | *"Johns Hopkins is located at the state"*) → "of"
6. Sample from **P**(X | *"Johns Hopkins is located at the state of"*) → "Maryland"
7. Sample from **P**(X | *"Johns Hopkins is located at the state of Maryland"*) → "EOS"

Notice this special word to indicate "end of sentences"

Why Care About Language Modeling?

- Language Modeling is a ~~subcomponent~~ superset of many tasks:
  - Summarization
  - Machine translation
  - Spelling correction
  - Dialogue etc.

- Language Modeling is an effective proxy for language understanding.
  - Effective ability to predict forthcoming words requires on understanding of context/prefix.

Summary

- **Language modeling:** building probabilistic distribution over language.
- An accurate distribution of language enables us to solve many important tasks that involve language communication.
- **Next question**: how do you actually estimate this distribution?

Language Modeling with Counting

LMs as a Marginal Distribution

$$\mathbf{P}(\overbrace{X_t}^{\text{next word}} \mid \overbrace{X_1, \ldots, X_{t-1}}^{\text{context}})$$

- Now the question is, how to estimate this distribution.

$$\mathbf{P}(X_t | X_1, \ldots, X_{t-1})$$

How do we estimate these probabilities?

Let's just count!

$$\mathrm{P}(\text{"mat"} \mid \text{"the cat sat on the"}) \approx \frac{\mathrm{count}(\text{"the cat sat on the mat"})}{\mathrm{count}(\text{"the cat sat on the"})}$$

Count how often
"the cat sat on the mat"
has appeared in the world (internet)!

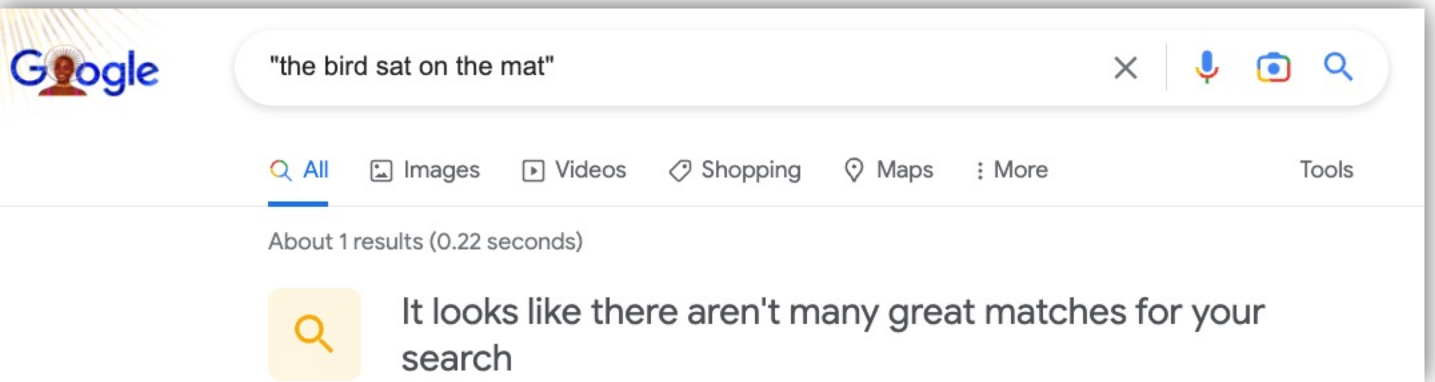
Divide that by, the count of
"the cat sat on the"
in the world (internet)!

$$\mathbf{P}(X_t | X_1, \ldots, X_{t-1})$$

How do we estimate these probabilities?

Let's just count!

$$P(\text{“mat”} \mid \text{“the cat sat on the”}) \approx \frac{\text{count(“the cat sat on the mat”)}}{\text{count(“the cat sat on the”)}}$$



28

$$\mathbf{P}(X_t | X_1, \ldots, X_{t-1})$$

How do we estimate these probabilities?

Let's just count!

$$\text{P(“mat” | “the cat sat on the”)} \approx \frac{\text{count(“the cat sat on the mat”)}}{\text{count(“the cat sat on the”)}}$$

<u>Challenge:</u> Increasing n makes sparsity problems worse.
Typically, we can't have n bigger than 5.

Some partial solutions (e.g., smoothing and backoffs) though still an open problem.

Language Models: A History

- Shannon (1950): The redundancy and predictability (entropy) of English.

Prediction and Entropy of Printed English

By C. E. SHANNON

(*Manuscript Received Sept. 15, 1950*)

A new method of estimating the entropy and redundancy of a language is described. This method exploits the knowledge of the language statistics possessed by those who speak the language, and depends on experimental results in prediction of the next letter when the preceding text is known. Results of experiments in prediction are given, and some properties of an ideal predictor are developed.

[Prediction and Entropy of Printed English, Shanon 1950]

$$\mathbf{P}(X_t \mid X_1, \ldots, X_{t-1})$$

Andrey Markov

Shannon (1950) built an approximate language model with word co-occurrences.

Markov assumptions: every node in a Bayesian network is conditionally independent of its non-descendants, given its parents.

1st order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \overbrace{\text{the}}^{1\ \text{element}})$$

[Prediction and Entropy of Printed English, Shanon 1950]

31

$$\mathbf{P}(X_t \mid X_1, \ldots, X_{t-1})$$

Andrey Markov

2nd order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \underbrace{\text{on the}}_{2 \text{ elements}})$$

1st order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \underbrace{\text{the}}_{1 \text{ element}})$$

[Prediction and Entropy of Printed English, Shanon 1950]

$$\mathbf{P}(X_t | X_1, \ldots, X_{t-1})$$

3rd order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \overbrace{\text{sat on the}}^{3 \text{ elements}})$$

2nd order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \overbrace{\text{on the}}^{2 \text{ elements}})$$

1st order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \overbrace{\text{the}}^{1 \text{ element}})$$

[Prediction and Entropy of Printed English, Shanon 1950]

$$\mathbf{P}(X_t \mid X_1, \ldots, X_{t-1})$$

Andrey Markov

3rd order approximation:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \underbrace{\text{sat on the}}^{\text{3 elements}})$$

Then, we can use counts of approximate conditional probability.
Using the 3rd order approximation, we can:

$$\mathbf{P}(\text{mat} \mid \text{the cat sat on the}) \approx \mathbf{P}(\text{mat} \mid \text{sat on the}) = \frac{\text{count(“sat on the mat”)}}{\text{count(“on the mat”)}}$$

[Prediction and Entropy of Printed English, Shanon 1950]

34

N-gram Language Models

- **Terminology:** n-gram is a chunk of n consecutive words:
  - unigrams: “cat”, “mat”, “sat”, …
  - bigrams: “the cat”, “cat sat”, “sat on”, …
  - trigrams: “the cat sat”, “cat sat on”, “sat on the”, …
  - four-grams: “the cat sat on”, “cat sat on the”, “sat on the mat”, …

- n-gram language model:

$$\mathrm{P}(X_t \mid X_1, \dots, X_{t-1}) \approx \mathrm{P}(X_t \mid \overbrace{X_{t-n+1}, \dots, X_{t-1}}^{n-1 \text{ elements}}) = \frac{\mathrm{count}(X_{t-n+1}, \dots, X_{t-1}, X_t)}{\mathrm{count}(X_{t-n+1}, \dots, X_{t-1})}$$

[Prediction and Entropy of Printed English, Shanon 1950]

How easy is it to build
an n-gram language model?

Generation from N-Gram Models

- You can build a simple **tri**gram Language Model over a 1.7 million words corpus in a few seconds on your laptop*

* Try for yourself: https://nlpforhackers.io/language-models/

[adopted from Chris Manning]

Generation from N-Gram Models

- You can build a simple **tri**gram Language Model over a 1.7 million words corpus in a few seconds on your laptop*

```
today the ____
```

get probability distribution

```
company   0.153
bank      0.153
price     0.077
italian   0.039
emirate   0.039
...
```

Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

* Try for yourself: https://nlpforhackers.io/language-models/

[adopted from Chris Manning]

Generation from N-Gram Models

- Now we can sample from this mode:

`today the ____`

get probability distribution

```
company   0.153
bank      0.153
price     0.077
italian   0.039
emirate   0.039
...
```

Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

* Try for yourself: https://nlpforhackers.io/language-models/

[adopted from Chris Manning]

Generation from N-Gram Models

- Now we can sample from this mode:

condition on this

$$\texttt{today } \underbrace{\texttt{the price}}_{}\ \texttt{\_\_\_\_\_}$$

get probability distribution

of	0.308
for	0.050
it	0.046
to	0.046
is	0.031
...	

Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

* Try for yourself: https://nlpforhackers.io/language-models/

[adopted from Chris Manning]

Generation from N-Gram Models

- Now we can sample from this mode:

condition on this

$$\texttt{today the } \underbrace{\texttt{price of}}\ \_\_$$

get probability distribution

the	0.072
18	0.043
oil	0.043
its	0.036
gold	0.018
...	

Sparsity problem: not much granularity in the probability distribution

Otherwise, seems reasonable!

* Try for yourself: https://nlpforhackers.io/language-models/

[adopted from Chris Manning]

N-Gram Models in Practice

- Now we can sample from this mode:

```
today the price of gold per ton , while production of shoe
lasts and shoe industry , the bank intervened just after it
considered and rejected an imf demand to rebuild depleted
european stocks , sept 30 end primary 76 cts a share .
```

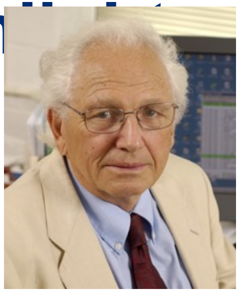
Surprisingly grammatical!

But quite incoherent! To improve coherence, one may consider increasing larger than 3-grams, but that would worsen the sparsity problem!

* Try for yourself: https://nlpforhackers.io/language-models/ [adopted from Chris Manning]

N-Gram Language Models, A Historical Highlight

"Every time I fire a linguist, the performance of the speech recognizer goes up"!!



Fred Jelinek
(1932-2010)

- Probabilistic n-gram models of text generation [Jelinek+ 1980's, …]
  - Applications: Speech Recognition, Machine Translation

532 PROCEEDINGS OF THE IEEE, VOL. 64, NO. 4, APRIL 1976

Continuous Speech Recognition by Statistical Methods

FREDERICK JELINEK, FELLOW, IEEE

Abstract—Statistical methods useful in automatic recognition of continuous speech are described. They concern modeling of a speaker and of an acoustic processor, extraction of the models' statistical parameters, and hypothesis search procedures and likelihood computations of linguistic decoding. Experimental results are presented that indicate the power of the methods.

utterance models used will incorporate more grammatical features, and statistics will have been grafted onto grammatical models. Most methods presented here concern modeling of the speaker's and acoustic processor's performance and should, therefore, be universally useful.

Automatic recognition of continuous (English) speech is an

Limits of N-Grams LMs: Long-range Dependencies

- In general, count-based LMs are insufficient models of language because language has long-distance dependencies:

"The computer which I had just put into the machine room on the fifth floor crashed."

Summary

- Learning a language model ~ learning conditional probabilities over language.
- One approach to estimating these probabilities: counting word co-occurrences.

- Challenges:
  - Word co-occurrences become rare for long sequences. (the sparsity issue)
  - But language understanding requires long-range dependencies.

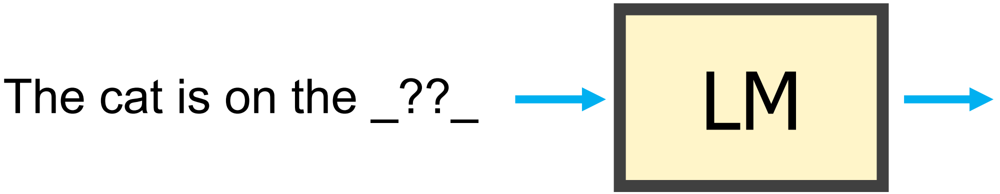
- We need a better alternative! 😤

- **Next:** Measuring quality of language models.

How Good are Language Models?

Large Language Models

- A language model can predict the next word based on the given context.

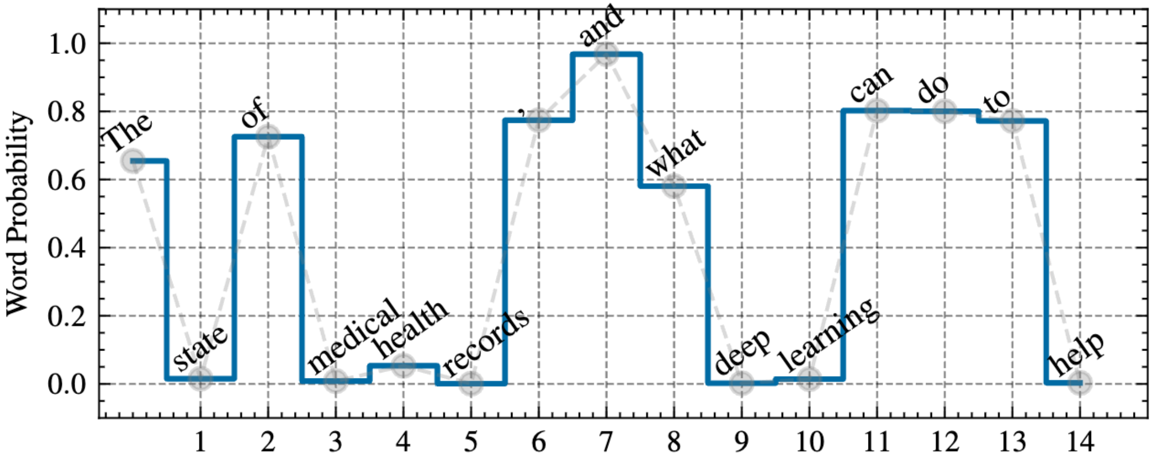


X := The cat is on the

roof	**P**(roof|X)=0.00
tree	**P**(tree|X)=0.01
moon	**P**(moon|X)=0.01
physics	**P**(physics|X)=0.1
the	**P**(the|X)=0.1
protein	**P**(protein|X)=0.3
…	**…**

An Example Next-Token Probabilities

- “The state of medical health records, and what deep learning can do to help”

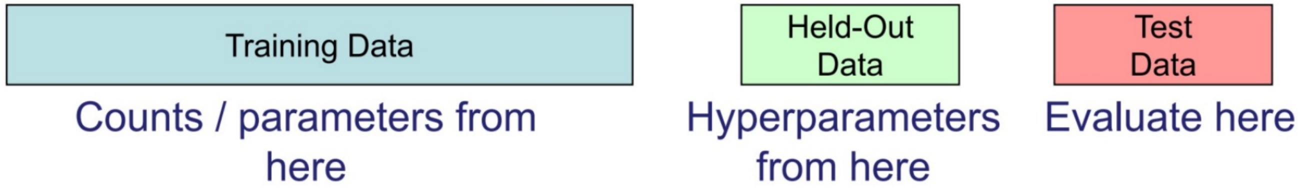


[Content Reduction, Surprisal and Information Density Estimation for Long Documents, 2023]()

Evaluating Language Models

Setup:

- Train it on a suitable training documents.
- Evaluate their predictions on different, unseen documents.
- An evaluation metric tells us how well our model does on the test set.



Quiz: Building Intuition

- Sample a sentence $(w_1, w_2, \ldots, w_n) = (\text{cat}, \text{sat}, \text{on}, \text{the}, \text{mat})$ from our natural data.
- We can show the probability that our language model assigns to this sentence with:

$$\mathbf{P}(w_1, w_2, \ldots, w_n)$$

- A strong language model would assign a __ probability to this sentence. (high or low?)

- A weak language model would assign a __ probability to this sentence. (high or low?)

Next, we will define "perplexity", a metric that quantifies LM's uncertainty with respect to a corpus of natural sentences.

Evaluation Metric for Language Modeling: Perplexity

- Sample a sentence $(w_1, w_2, \dots, w_n)$ from our natural data.
- **Perplexity** is the inverse probability of the test set, normalized by the number of words:

$$\mathrm{ppl}(w_1, \dots, w_n) = \mathbf{P}(w_1, w_2, \dots, w_n)^{-\frac{1}{n}}$$

The negative power $(.)^{-}$ inverses the score. So, a small probability become a larger score – working with small numbers is tedious.

$\frac{1}{n}$ normalizes the probability as a function of length so that longer sequences are not assigned lower scores.

- A measure of predictive quality of a language model.
- A LM with lower perplexity is better because it assigns a higher probability to the unseen test corpus.

Evaluation Metric for Language Modeling: Perplexity

- Sample a sentence $(w_1, w_2, \dots, w_n)$ from our natural data.
- **Perplexity** is the inverse probability of the test set, normalized by the number of words:

$$\mathrm{ppl}(w_1, \dots, w_n) = \mathbf{P}(w_1, w_2, \dots, w_n)^{-\frac{1}{n}}$$

But wait, we usually have conditionals not the joint distribution! 🤔

Evaluation Metric for Language Modeling: Perplexity

- Sample a sentence $(w_1, w_2, \dots, w_n)$ from our natural data.
- **Perplexity** is the inverse probability of the test set, normalized by the number of words:

$$\mathrm{ppl}(w_1, \dots, w_n) = \mathbf{P}(w_1, w_2, \dots, w_n)^{-\frac{1}{n}}$$

$$= \sqrt[n]{\frac{1}{\mathbf{P}(w_1, w_2, \dots, w_n)}} = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{\mathbf{P}(w_i | w_{<i})}} \quad \text{chain rule}$$

$$= 2^{H}, \text{where}$$

$$H = -\frac{1}{n} \sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \dots, w_{i-1})$$

Putting Things Together: Perplexity Definition

- For a given a sampled sentence $(w_1, w_2, \dots, w_n)$ from our natural data:

$$\text{ppl}(w_1, \dots, w_n) = 2^H, \text{ where } H = -\frac{1}{n}\sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \dots, w_{i-1})$$

- Notice that this consists of probability assigned to all the partial sentences (i.e., next word probabilities).

- In practice, we prefer to use log-probabilities (also known as “logits”) since probabilities are too small and hard to understand (e.g., 10^-18 vs -18).

Intuition-building Quizzes (1)

- **Quiz:** let’s we evaluate a confused (!!) model of language, i.e., our model has no idea what word should follow each context—it always chooses a uniformly random word. What is the perplexity of this model?
- Answer: $|V|$ (size of the vocabulary) – why?

Intuition-building Quizzes (1)

- **Quiz:** let's we evaluate a confused (!!) model of language, i.e., our model has no idea what word should follow each context—it always chooses a uniformly random word. What is the perplexity of this model?
- Sample a sentence from corpus: X="The cat is on the mat."

For any partial sub-sentence:
X=The cat is on the _??_ → LM →

roof
tree
moon
physics
…

P(roof|X)=1/|V|
P(tree|X)=1/|V|
P(moon|X)=1/|V|
P(physics|X)=1/|V|
…

$$\forall w \in V: \ \mathbf{P}(w|w_{1:i-1}) = \frac{1}{|V|} \ \Rightarrow \ \mathrm{ppl}(D) = 2^{-\frac{1}{n} n \log_2 \frac{1}{|V|}} = |V|$$

Intuition-building Quizzes (2)

- **Quiz:** let's suppose we have a sentence $w_1, \dots, w_n$ and it's fixed. Our language is model is mildly confused because it narrows down the plausible continuations to 5 words, but it is confused among them. So it it assigns probability 1/5 to the correct next word. What is perplexity of our model?

A partial sentence:
X=The cat is on the _??_ → LM →

P(roof|X)=1/5
P(tree|X)=1/5
P(table|X)=1/5
P(mat|X)=1/5
P(wall|X)=1/5
P(physics|X)=0
P(tesla|X)=0
…

Our LM has narrowed down the right continuation to one of these five words.

$$H = -\frac{1}{n}\left[\log_2\left(\frac{1}{5}\right) + \cdots + \log_2\left(\frac{1}{5}\right)\right] = -\log\left(\frac{1}{5}\right) \Rightarrow \mathrm{ppl(D)} = 5$$

Intuition: the model is indecisive among 5 choices.

Intuition-building Quizzes (3)

- **Quiz:** let's we evaluate an exact (!!) model of language, i.e., our model always knows what exact word should follow a given context. What is the perplexity of this model?

A partial sentence:
X=The cat is on the _??_ → **LM** →

P(mat|X)=1
P(physics|X)=0
P(tesla|X)=0
...

$$\forall w \in V: \mathbf{P}(w_i | w_{1:i-1}) = 1 \;\Rightarrow\; \text{ppl}(D) = 2^{-\frac{1}{n} n \log_2 1} = 1$$

Intuition: the model is indecisive among 1 (the right!) choice!

Perplexity: Summary

$$\text{ppl}(w_1, \ldots, w_n) = 2^H, \text{ where } H = -\frac{1}{n}\sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \ldots, w_{i-1})$$

- Perplexity is a measure of model's uncertainty about next word (aka "average branching factor").
  - The larger the number of vocabulary, the more options there to choose from.
  - (the choice of atomic units of language impacts PPL — more on this later)

- [The empirical estimates of] Perplexity ranges between 1 and |V|.

- We prefer LMs with lower perplexity.

- Next let's see more insights about PPL.

A note about terminology: Bits vs Nats

- Remember previous we defined PPL as:

$$\text{ppl}(w_1, \ldots, w_n) = 2^H, \text{ where } H = -\frac{1}{n}\sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \ldots, w_{i-1})$$

- A common name for H is "loss" (a function that we want to minimize).
- The unit of loss depends the base of the log.
- The unit of this definition (which uses base-2) is "bits".
- If you use base-e (Napier constant), then unit becomes "nats".
- So, both nats & bits are units of information/entropy.

Perplexity as An Implicit Cross-Entropy

- Compare the definition of PPL and cross-entropy:

$$\text{ppl}(w_1, \ldots, w_n) = 2^H, \text{ where } H = -\frac{1}{n}\sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \ldots, w_{i-1})$$

Cross-entropy between two distributions q, p :

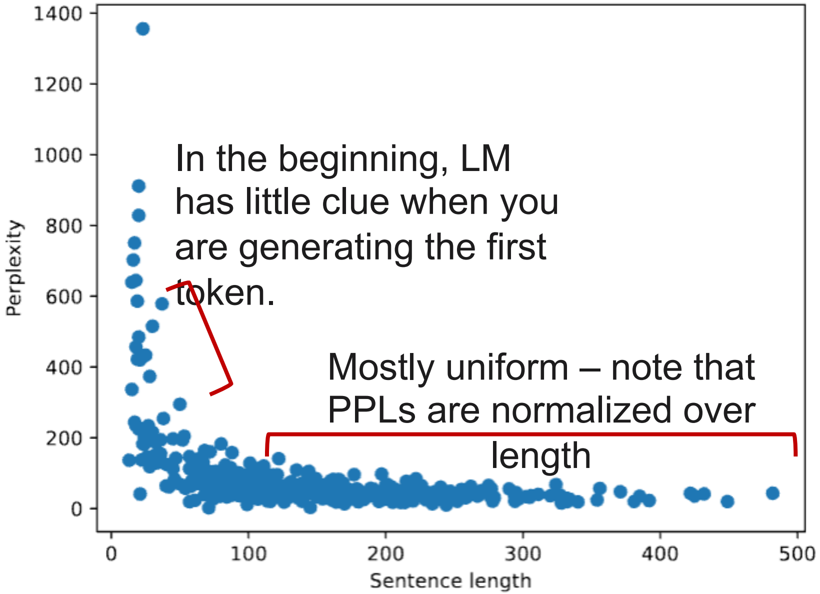
$$\text{CE}(q, p) = -\sum_{x \in \mathcal{X}} q(x) \log p(x)$$

The ***H*** term in PPL can be interpreted as cross-entropy between LM distribution and the latent (unobserved) distribution of language. **Why?**
Hint: Monte Carlo Approximation + Law of Large Numbers.

Quiz: Thinking About Sentence Lengths

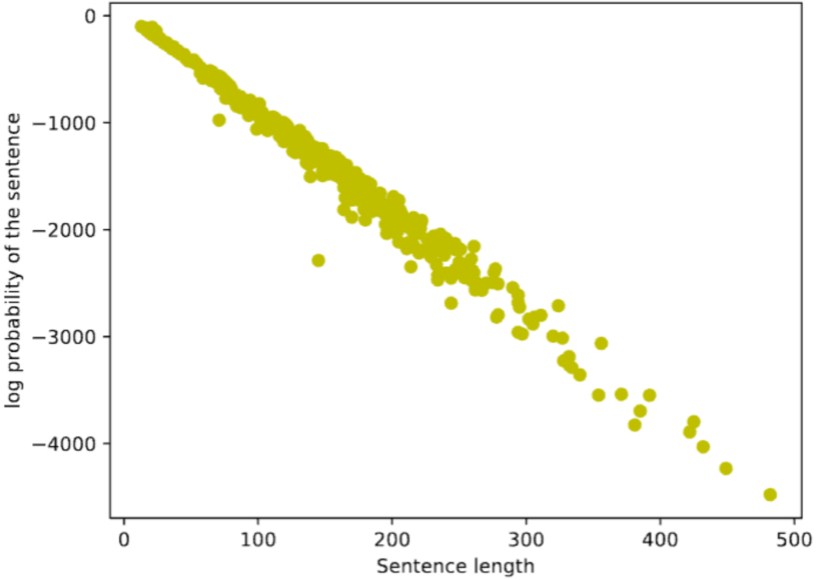
- Would you expect higher PPL for longer sentences?

$$\text{ppl}(w_1, \ldots, w_n) = 2^{-\frac{1}{n}\sum_{i=1}^{n} \log_2 \mathbf{P}(w_i | w_1, \ldots, w_{i-1})}$$



- Would you expect a higher probability for longer sentences?

$$\mathbf{P}(w_1, \ldots, w_n) = \prod_{i=1}^{n} \mathbf{P}(w_i | w_1, \ldots, w_{i-1})$$



Lower perplexity == Better Model

- Training on 38 million words, test 1.5 million words, Wall Street Journal

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

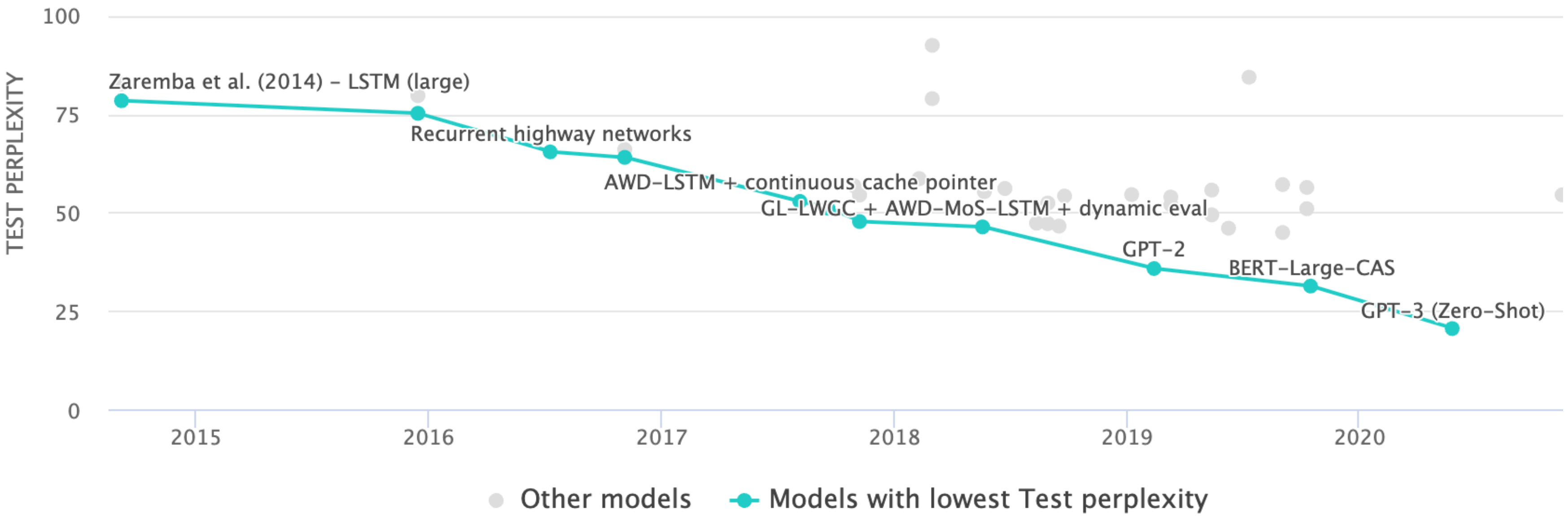
Lower is better

Note these evaluations are done on data that was not used for "counting." (no cheating!!)

[Mohit Iyyer]

Lower perplexity == Better Model

The PPL of modern language models have consistently been going down.



[Language Modelling on Penn Treebank (Word Level)]

Summary

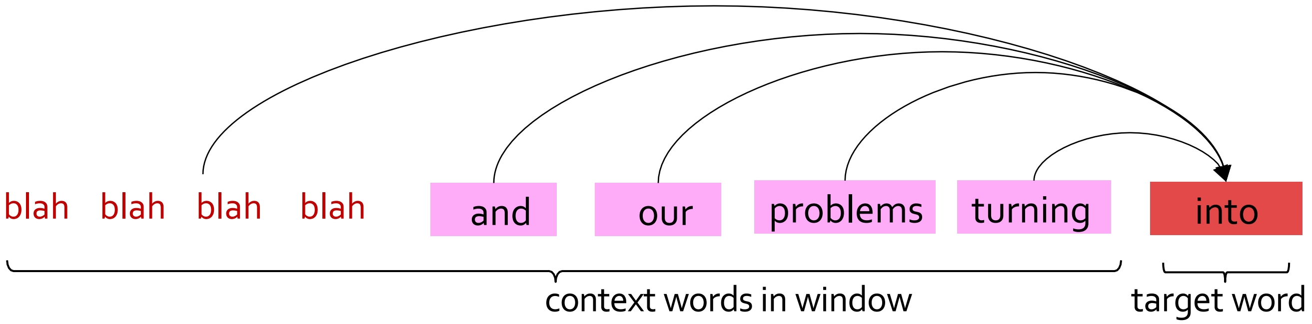
- PPL is length normalized.
- The *H* terms (when computing PPL) can be thought of a cross-entropy between two distributions.
- An [over]simplified history of AI: lower PPL => stronger intelligence.
- Next: Rethinking language modeling as a statistical learning problem.

Language Modeling as a Machine Learning Problem

LM as a Machine Learning Problem

- Given the embeddings of the context, predict the word on the right side.
  - Dropping the right context for simplicity -- not a fundamental limitation.

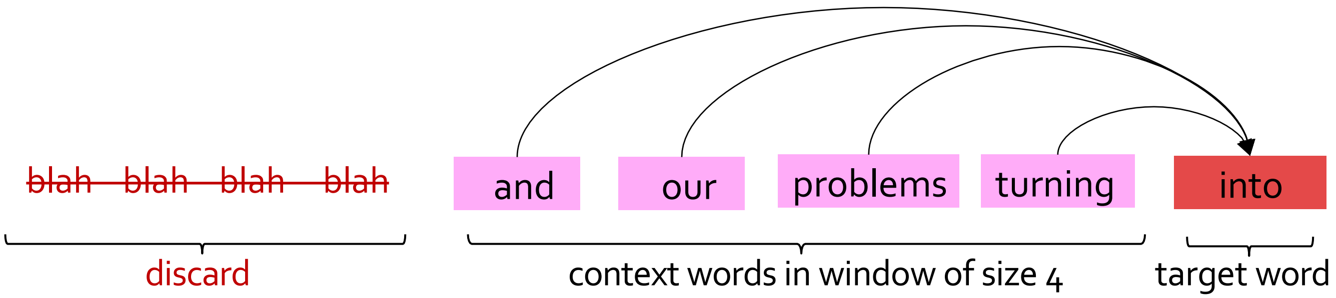
- Discard anything beyond its context window



LM as a Machine Learning Problem

- Given the embeddings of the context, predict the word on the right side.
  - Dropping the right context for simplicity -- not a fundamental limitation.

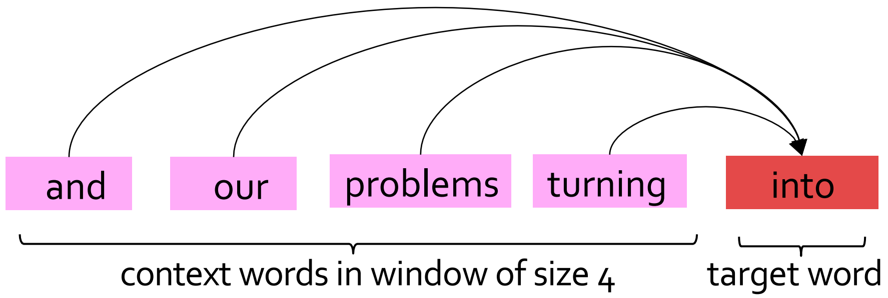
- Discard anything beyond its context window



LM as a Machine Learning Problem

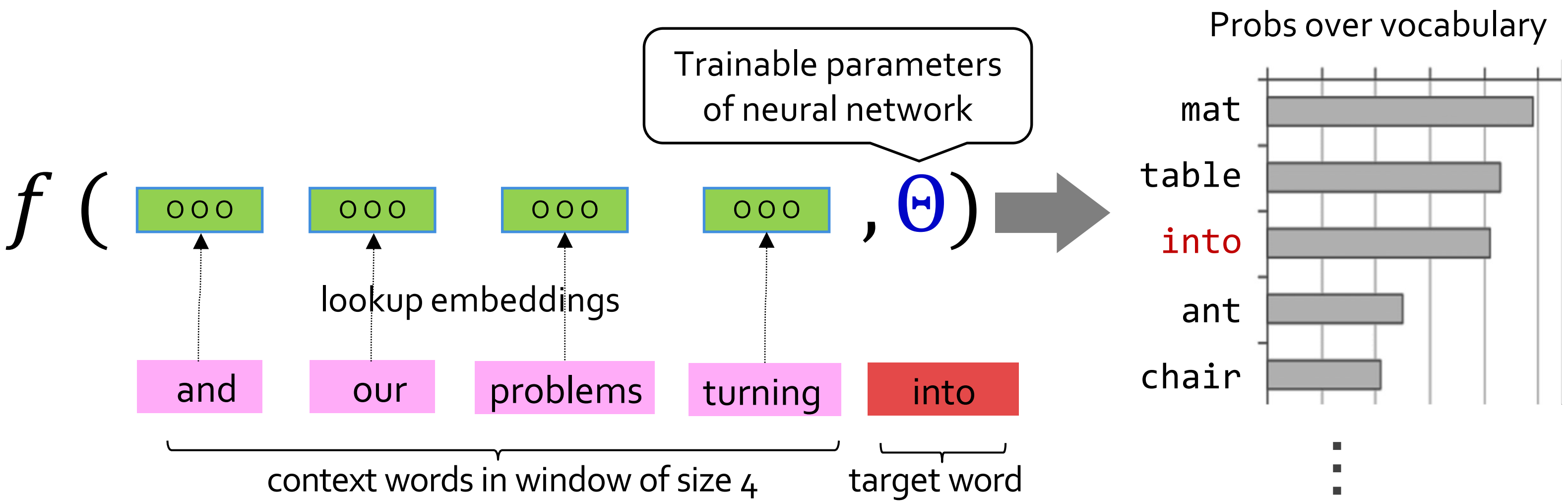
- Given the embeddings of the context, predict the word on the right side.
  - Dropping the right context for simplicity -- not a fundamental limitation.

- Discard anything beyond its context window



A Fixed-Window Neural LM

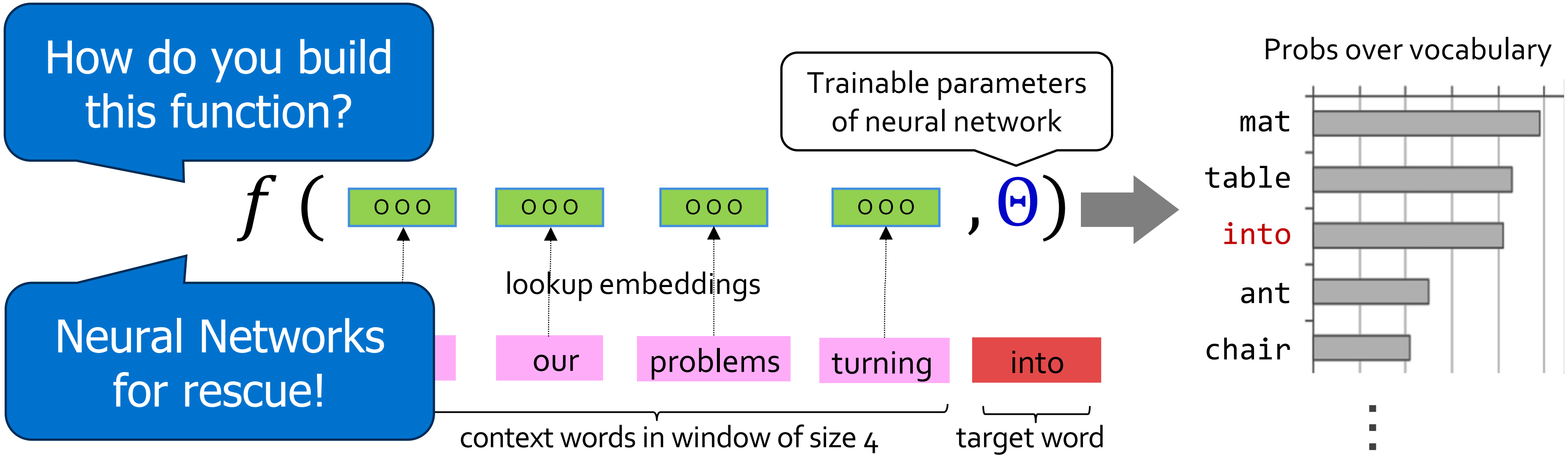
- Given the embeddings of the context, predict a target word on the right side.
  - Dropping the right context for simplicity -- not a fundamental limitation.
- Training this model is basically optimizing its parameters Θ such that it assigns high probability to the target word.



[Bengio et al. 2003]

A Fixed-Window Neural LM

- It will also lay the foundation for the future models (recurrent nets, transformers, ...)
- But first we need to figure out how to train neural networks!



[Bengio et al. 2003]

From Counting (N-Gram) to Neural Models

- N-gram models (~1980 to early 2000's),
  - Early instances of LMs
  - We saw usefulness of n-gram models for text generation [Jelinek+ 1980's, …]
  - In applications such as Speech Recognition, Machine Translation
  - Difficult to scale to large window sizes

From Counting (N-Gram) to Neural Models

- N-gram models (~1980 to early 2000's),
- Shallow neural LMs (early and mid-2000's) [Bengio+ 1999 & 2001, …]
  - These become more effective predictive models

A Neural Probabilistic Language Model

NeurIPS 2000

Yoshua Bengio*, Réjean Ducharme and Pascal Vincent
Département d'Informatique et Recherche Opérationnelle
Centre de Recherche Mathématiques
Université de Montréal
Montréal, Québec, Canada, H3C 3J7
{bengioy,ducharme,vincentp}@iro.umontreal.ca

- We will learn about build neural networks in the next chapter.